SMARTwinFA

how everything works on the server

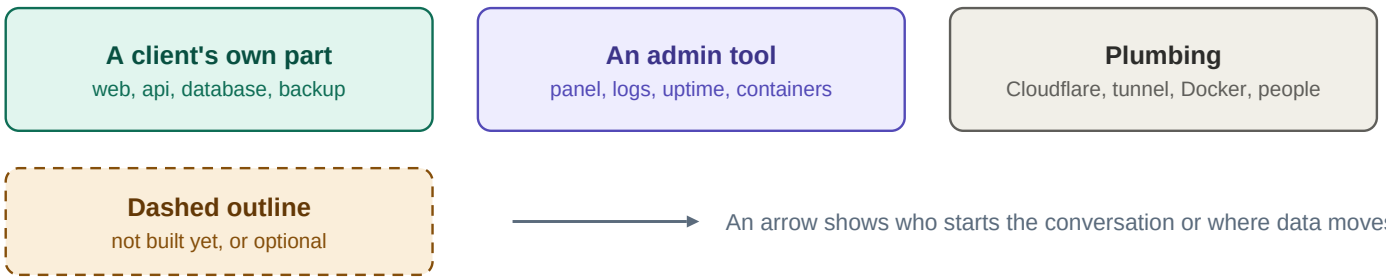
A picture-first guide for the owner: what runs where, how people sign in, how logs and backups work, and how a change gets from the code to the server.

Rishabh Plastic is the first client. Status as of 27 October 2026.

What is inside

1	**The big picture**	Who connects, through what, to what
2	**One client's stack**	The four parts every client gets
3	**Opening the app**	What happens from the address bar to the data
4	**Users and invite links**	How people get a login and a password
5	**Legacy and Modern views**	The remembered screen style
6	**The admin side**	One address for the four admin tools
7	**How logs travel**	From a container to a chart you can read
8	**The control panel**	Cards, colours and the two buttons
9	**Backups**	Nightly, manual, and restoring
10	**How a change reaches the server**	Code, checks, build, rollback
11	**Adding another client**	Eight steps
12	**Where things live, and what to check**	Locations, passwords, troubleshooting
13	**What is done and what is open**	Your to-do list and ours

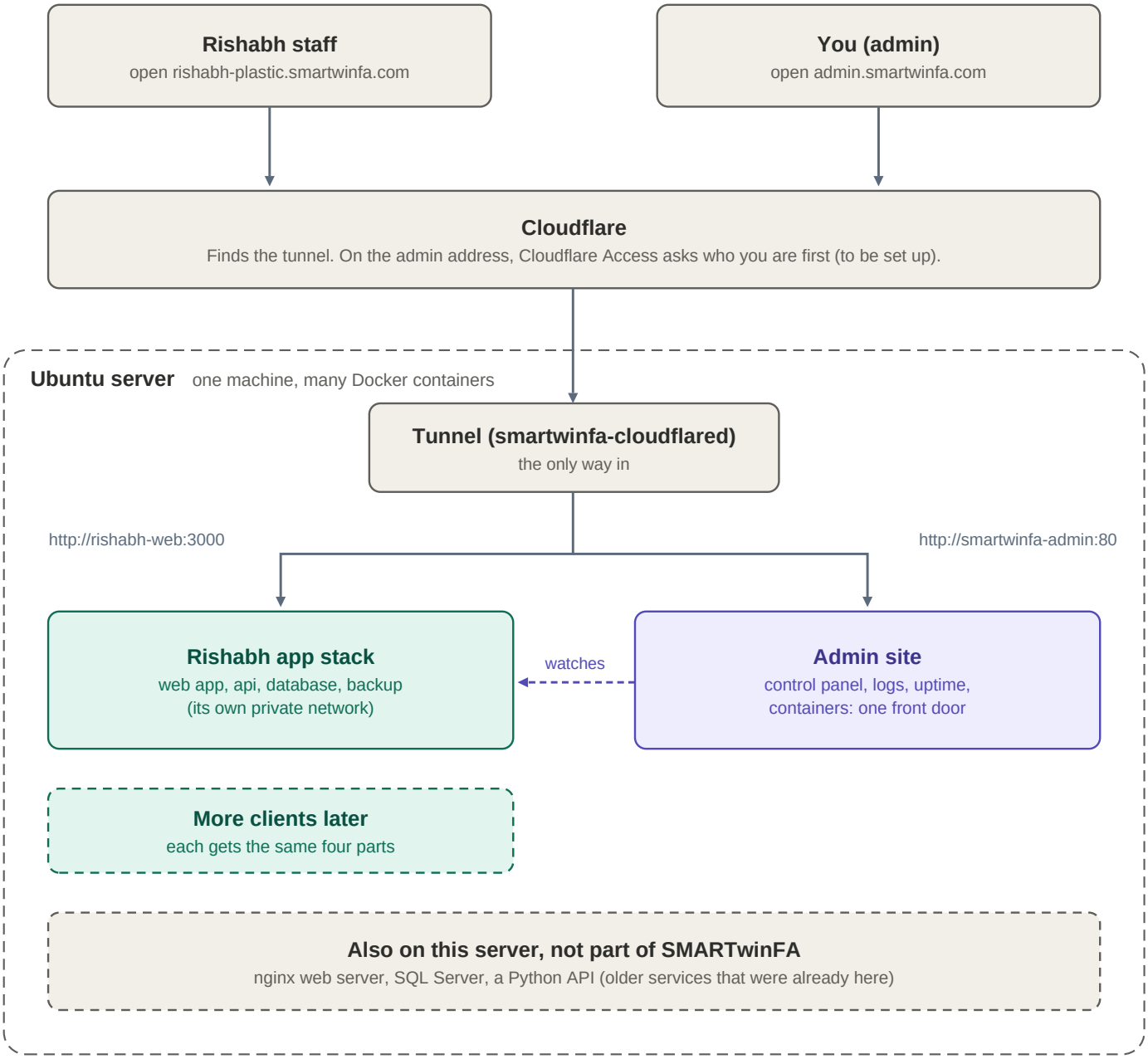
How to read the diagrams



Names in monospace, such as rishabh-web, are the real container, file or command names.

1 The big picture

Everything runs on one Ubuntu server, inside Docker containers. People never connect to the server directly: they go through Cloudflare and a single tunnel.

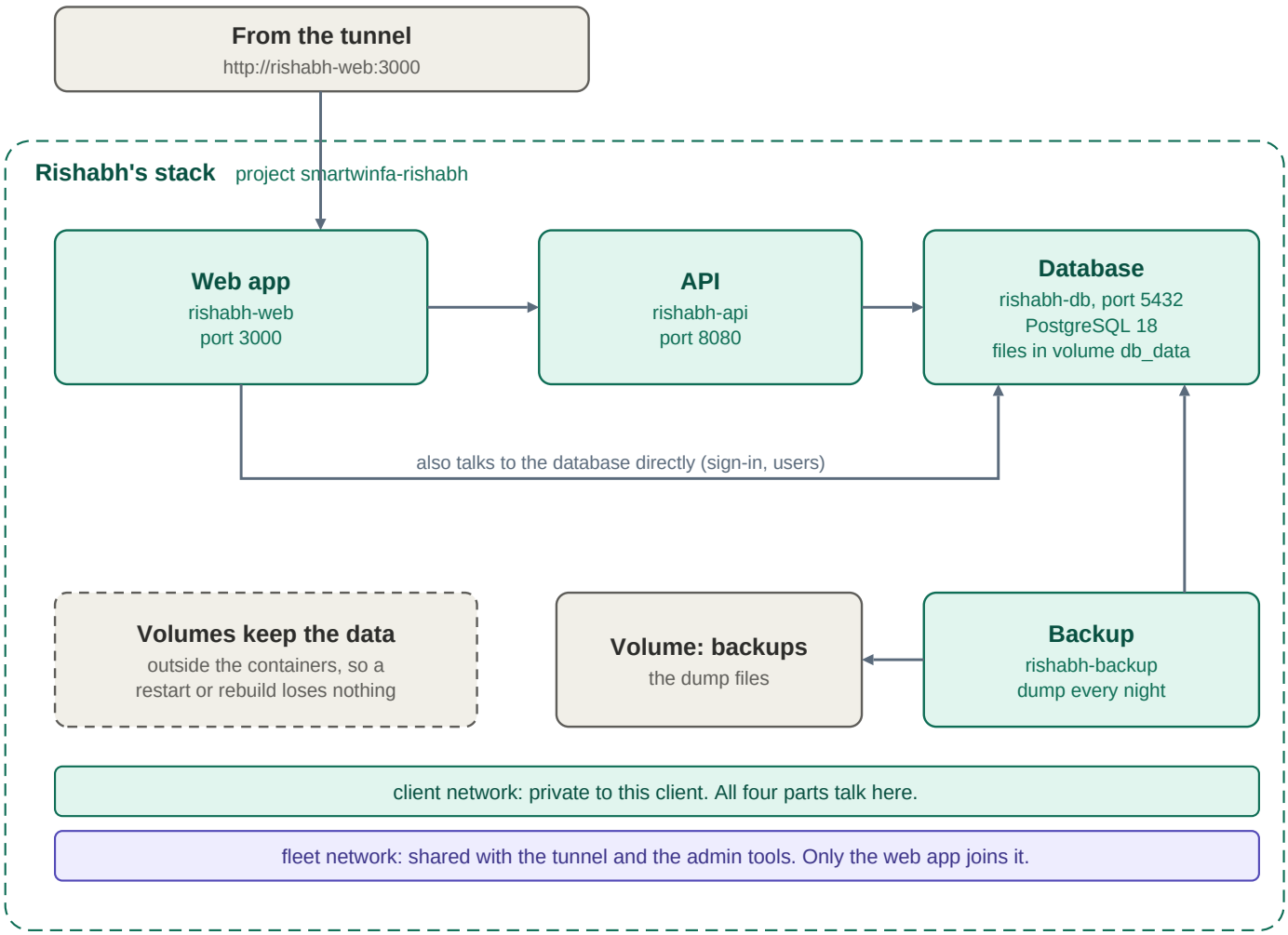


Three things to remember

- **Everything is a container.** The app, its database, the tunnel and the admin tools are all Docker containers. They find each other by name, such as `rishabh-web`.
- **One door.** The tunnel is the only way in from the internet, so the server needs no open ports for the app. Port numbers like 9090 or 4101 exist only on the server itself.
- **One stack per client.** A client's data never mixes with another's: each has its own database and its own private network. Only the admin tools see across clients.

2 One client's stack

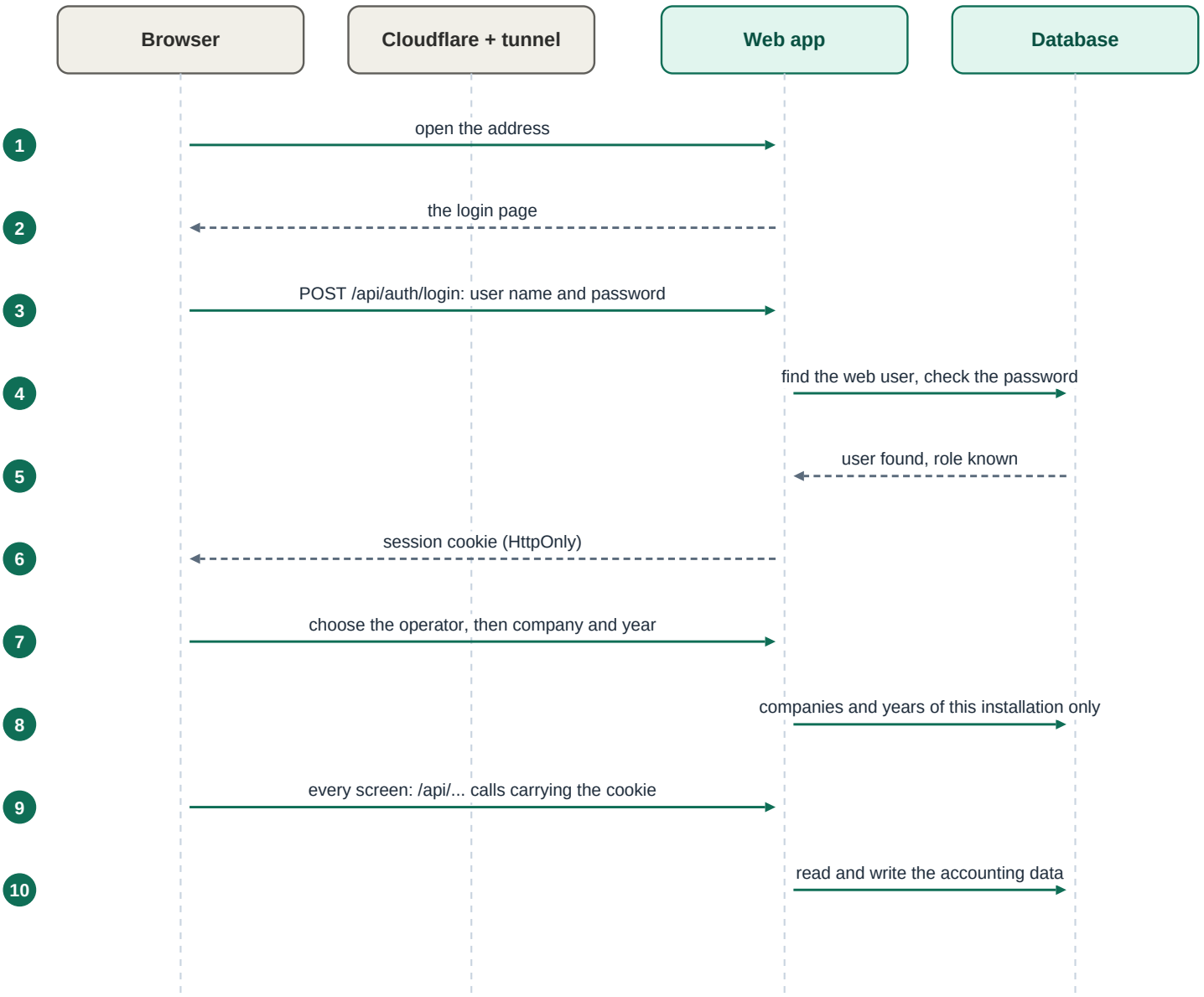
Every client gets the same four containers, started together. Rishabh's are named `rishabh-web`, `-api`, `-db` and `-backup`.



Part	What it does	Name	Reachable at
Web app	The screens people use, sign-in and sessions, reports. Reads data through the API and the database.	`rishabh-web`	3000 (4101 on the server itself)
API	Reads the accounting data on the web app's behalf. It is the old desktop program's logic, ported.	`rishabh-api`	8080, inside only
Database	PostgreSQL holding this client's books, plus the web users and sessions.	`rishabh-db`	5432, inside only
Backup	Writes a compressed dump of the database every night (and when asked).	`rishabh-backup`	none

3 Opening the app

What happens between typing the address and seeing your data. The tunnel sits between the browser and the web app; it only passes traffic along.

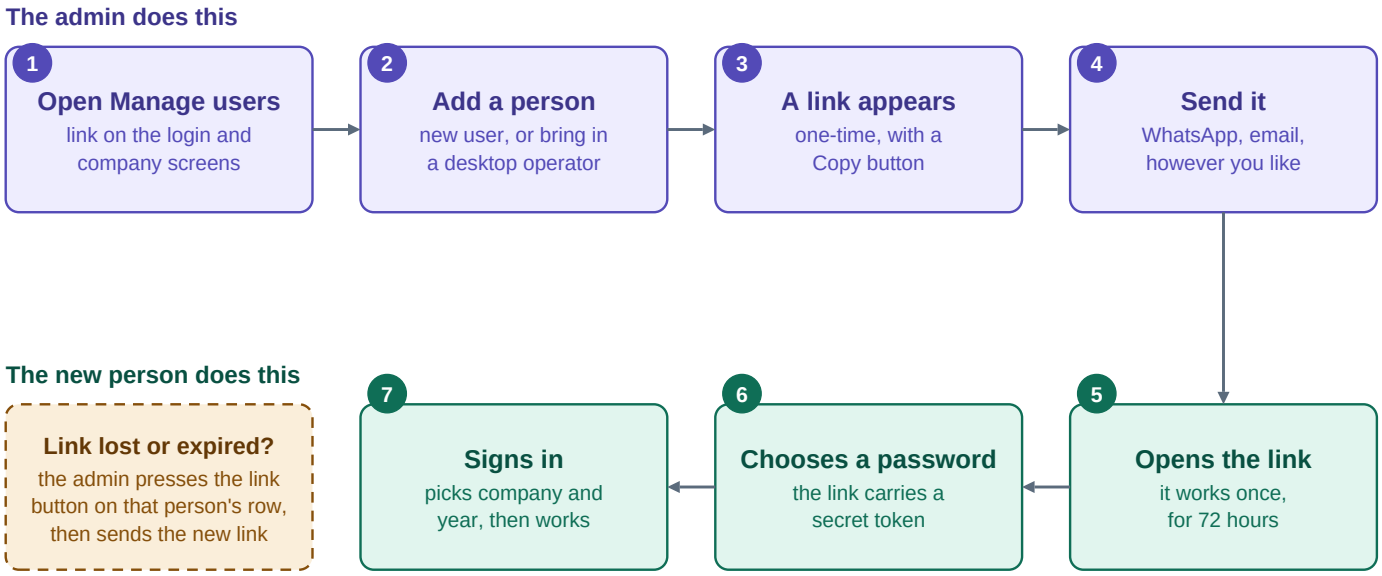


In plain words

- **Steps 3 to 6 are the sign-in.** The password is checked against a stored hash, never the password itself. The browser keeps a cookie that proves you signed in.
- **Step 7 depends on your role.** A platform admin picks which desktop operator to work as; everyone else is tied to their own operator. That operator's company, year and menu rights apply.
- **Without the cookie, nothing opens.** Every /api call is refused (401) except sign-in, set-password and the health check. A change sent from another website is refused too.
- **The web app and the API both use the same database.** Reports go through the API; sign-in and users are read directly.

4 Users and invite links

Nobody picks their own password in the open. An admin adds the person, the app produces a one-time link, and the admin sends it. Nothing is emailed by the app yet.



The very first admin of a server

no admin exists yet, so it comes from one command on the server:
docker exec rishabh-web node scripts/auth-invite.mts <email> --url https://<address>

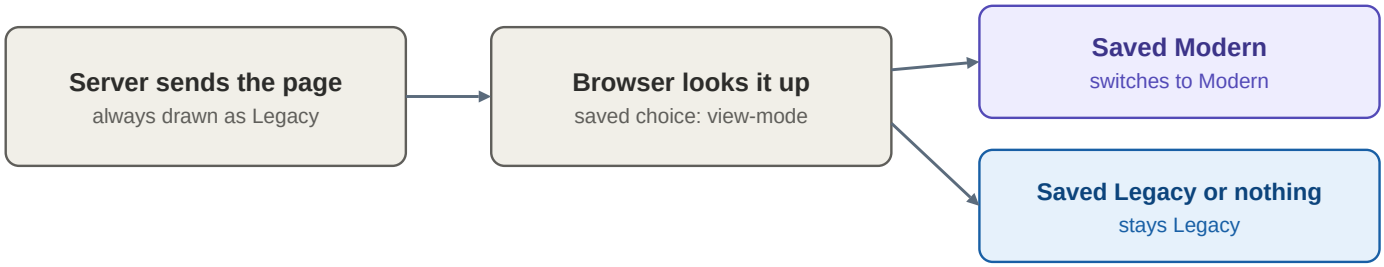
Role	Can manage	Works as	Typical person
Platform admin	Everyone	Chooses an operator each time they sign in	You (the vendor)
Company admin	Everyone except platform admins	Their own operator	The client's boss
Employee	No one	Their own operator	Day-to-day staff

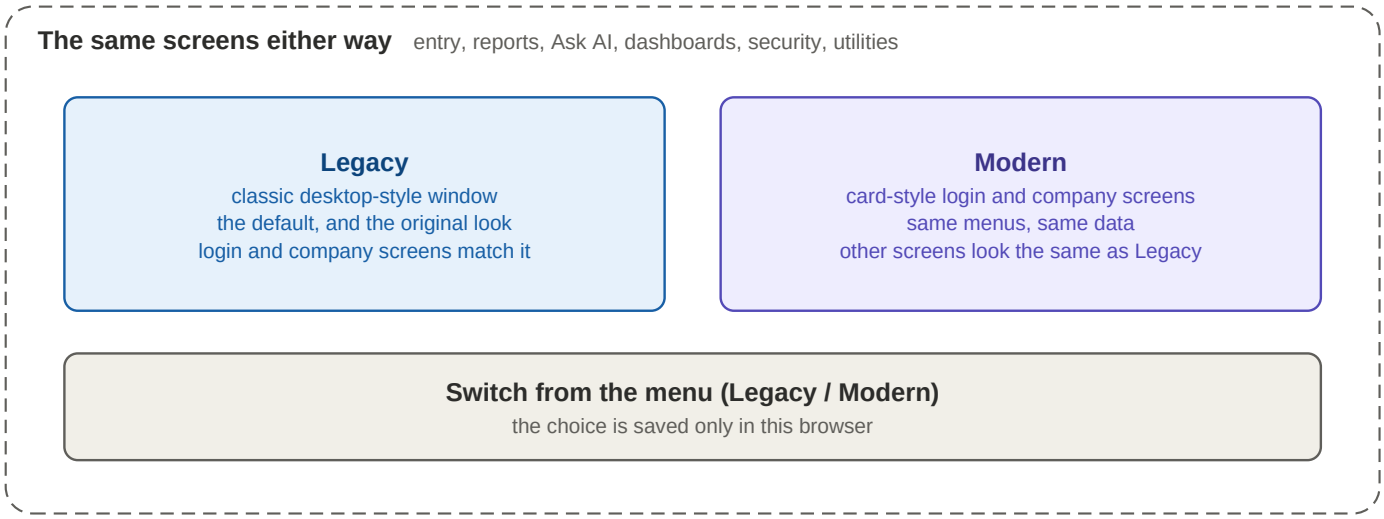
Good to know

- **Invite links last 72 hours, password-reset links 24 hours.** A new link cancels the old one. The link keeps its secret after a # so it is not sent to servers or written in their logs.
- **Disabling a user signs them out everywhere.** Their account stays, so it can be enabled again.
- **Two kinds of people exist.** A *desktop operator* is the client's login from the desktop program (ADMIN, PRANAV, SANDEEP, SIESTA at Rishabh) and comes from their own database. A *web user* is the person who signs in on the website and is tied to one operator.
- **Emailing the link is not built yet.** A task to add it was proposed and has not been started.

5 Legacy and Modern views

Two coats of paint over the same screens. The app remembers your choice in each browser, so it survives reloads and the next sign-in.



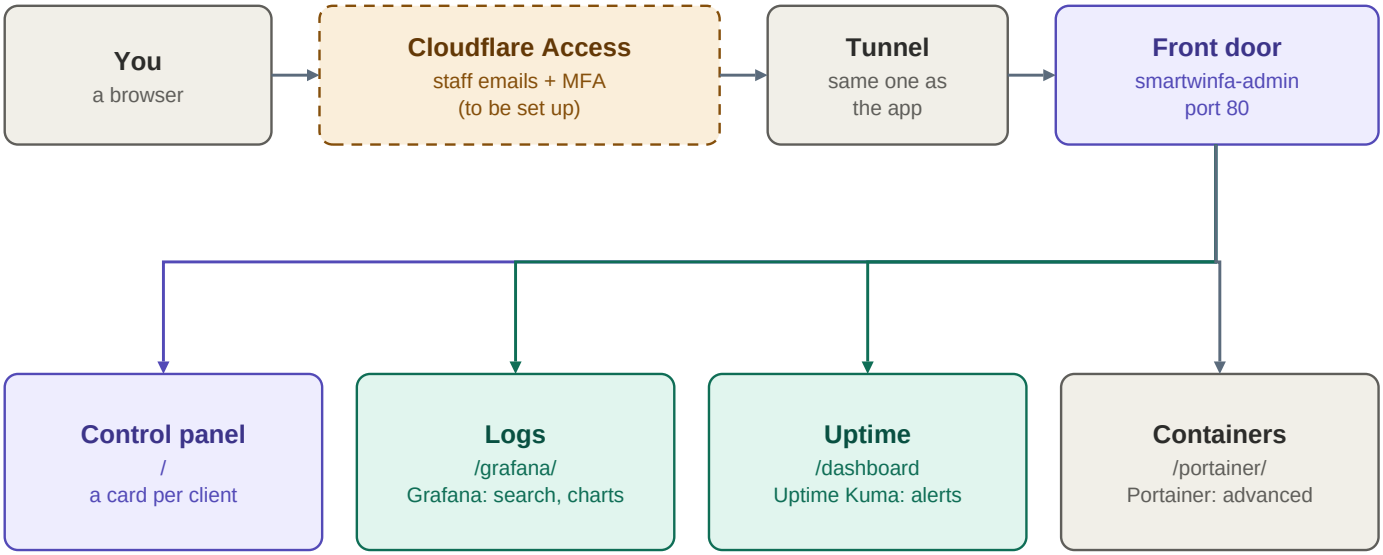


How it behaves

- **Per browser, not per user.** Your laptop and an office PC can show different views for the same login. Other tabs of the same browser follow a switch at once.
- **The page starts as Legacy for a moment,** then switches if Modern was saved. That keeps the page the server sends identical for everyone.
- **If the browser blocks storage** (a private window, for example), the choice lasts only until the page is closed.
- **Almost all October screen changes are in both views.** Only the window background, the strip colour under the title, the Add-on master form and the login and company screens differ.
- **The Legacy / Modern button was removed from the login screens** on 21 October. The menu is where you change it.

6 The admin side

One address, admin.smartwinfa.com, opens a front door that leads to four tools. You sign in to Cloudflare Access first (to be set up), then to each tool.



Tool	Use it to	Sign-in
Control panel `/`	See every client as a card. Open one for its parts, errors, backups and logs. Restart the app, or back up now.	Its own password (`PANEL_PASSWORD`), user not needed
Grafana `/grafana/`	Search logs over days, chart errors, open the SMARTwinFA Clients dashboard. Use *Explore* for your own queries.	User `admin` and its password
Uptime Kuma `/dashboard`	Get an alert when an address stops answering. Add one monitor per client: `http://rishabh-web:3000/api/health`.	You create the admin on the first visit
Portainer `/portainer/`	Advanced: a console inside a container, images, volumes. Powerful, so use it rarely.	User `admin` and its password

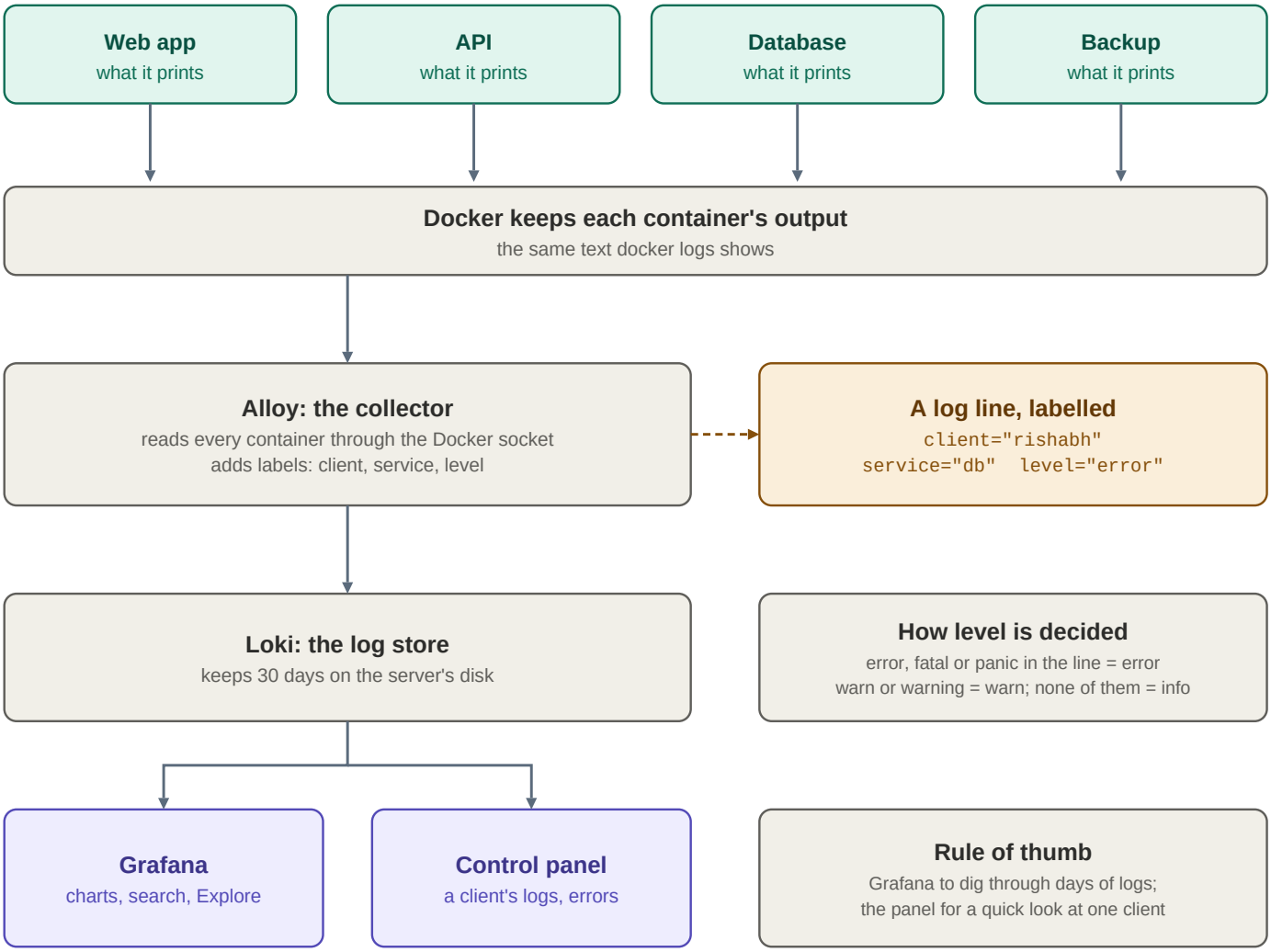
Good to know

- **The front door does not add a password of its own.** Portainer and Grafana send their own sign-in header, and a second password in front would block them. Cloudflare Access is the lock for the whole address.
- **Until the Cloudflare address exists,** reach it from your Mac through an SSH tunnel and open http://localhost:9090:

  ```
  ssh -p 57289 -N -L 9090:127.0.0.1:9090 ubuntu@5.175.139.120
  ```

7 How logs travel

Every container already writes what it prints to Docker. A collector copies it, labels it by client, and stores it so you can search it later.

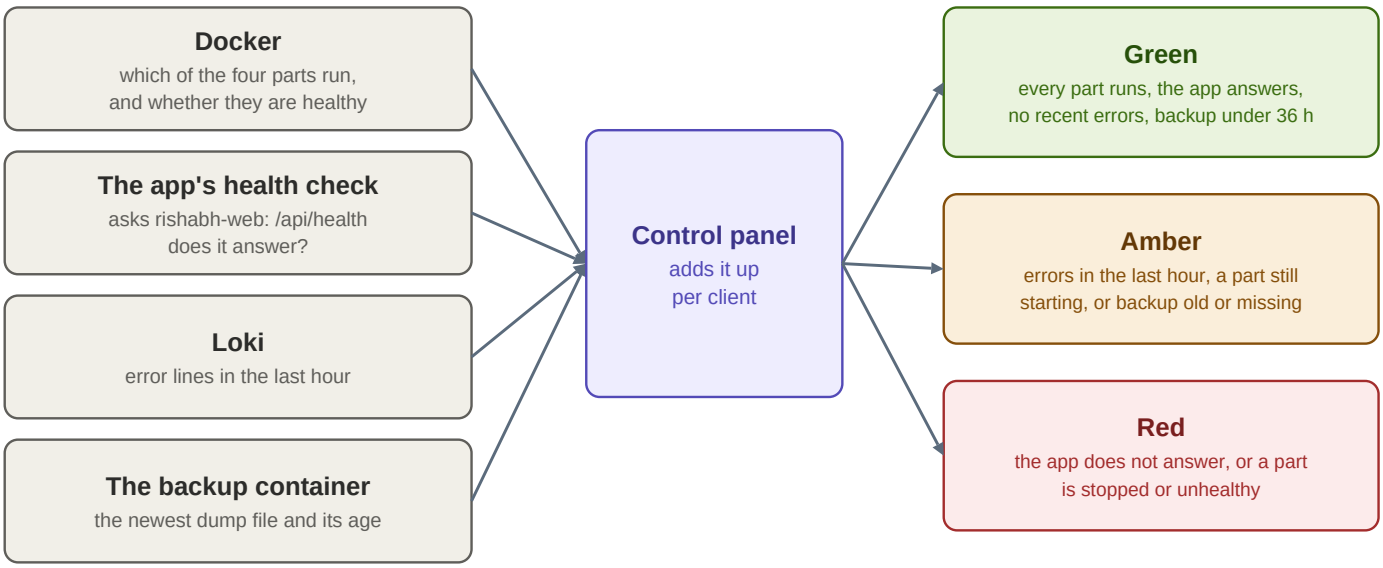


Why the web app looks quiet

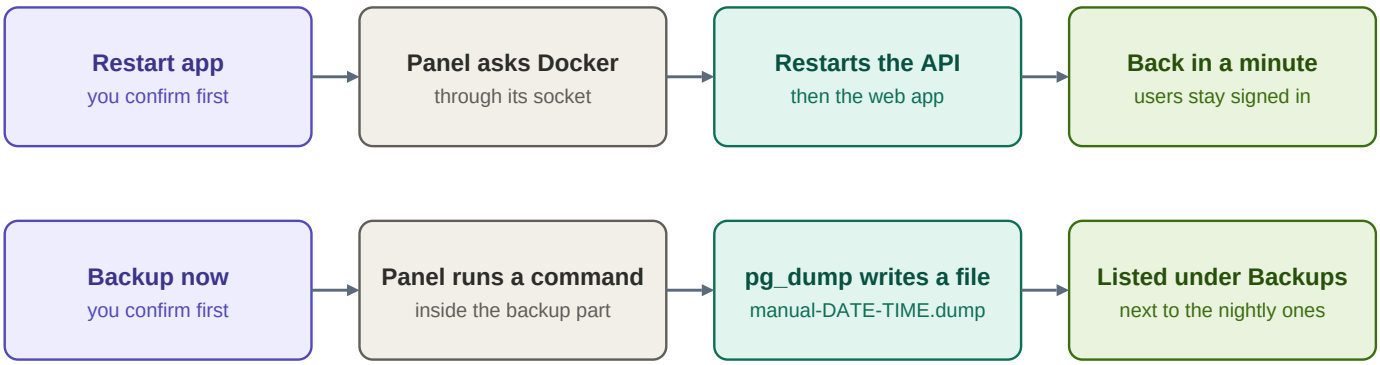
- **The web app and the API print almost nothing** unless something starts or fails. At Rishabh, seven days held 8 lines from the web app and 2 from the API. Nothing is wrong: the collector copies whatever the app prints, and the app prints little.
- **The database and the backup do print:** checkpoints and any query error, and a line when a backup finishes. Those are most of what you will see today.
- **A proper request log** (who did what, how long it took) is on the to-do list. Until then, Grafana is best for database errors and for checking that nothing restarted.
- **New clients need nothing:** a container with the label `smartwinfa.client` is picked up by itself within 15 seconds.

8 The control panel

The home page of the admin site. Each card gathers four facts about a client and turns them into one colour you can read from across the room.



The two buttons on a client's page

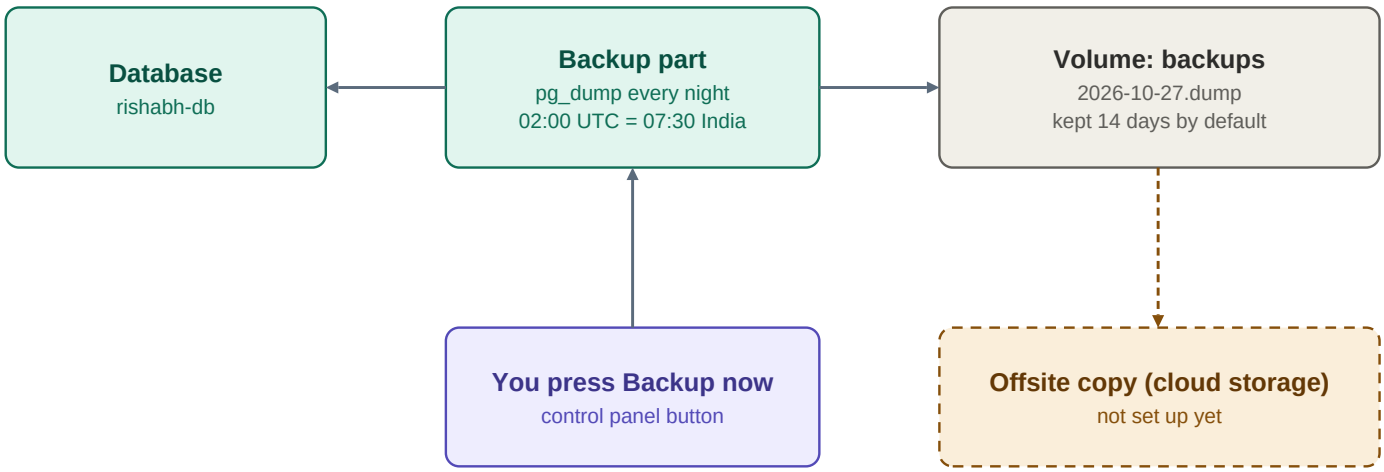


How it is kept safe

- **Its own password,** then a cookie that lasts 12 hours and ends if the panel restarts. Ten wrong passwords in ten minutes pauses sign-in for a while.
- **Changes need a special header** that a page on another website cannot send, so a booby-trapped page cannot press Restart for you.
- **One job at a time per client:** pressing Backup twice while it runs is refused.
- **It can control every container** (it holds the Docker socket), which is why the address must sit behind Cloudflare Access. It uses about 20 to 60 MB of memory.

9 Backups

A safety copy of each client's database is written every night, and on demand. Today these copies stay on the same server; copying them off it is not built yet.



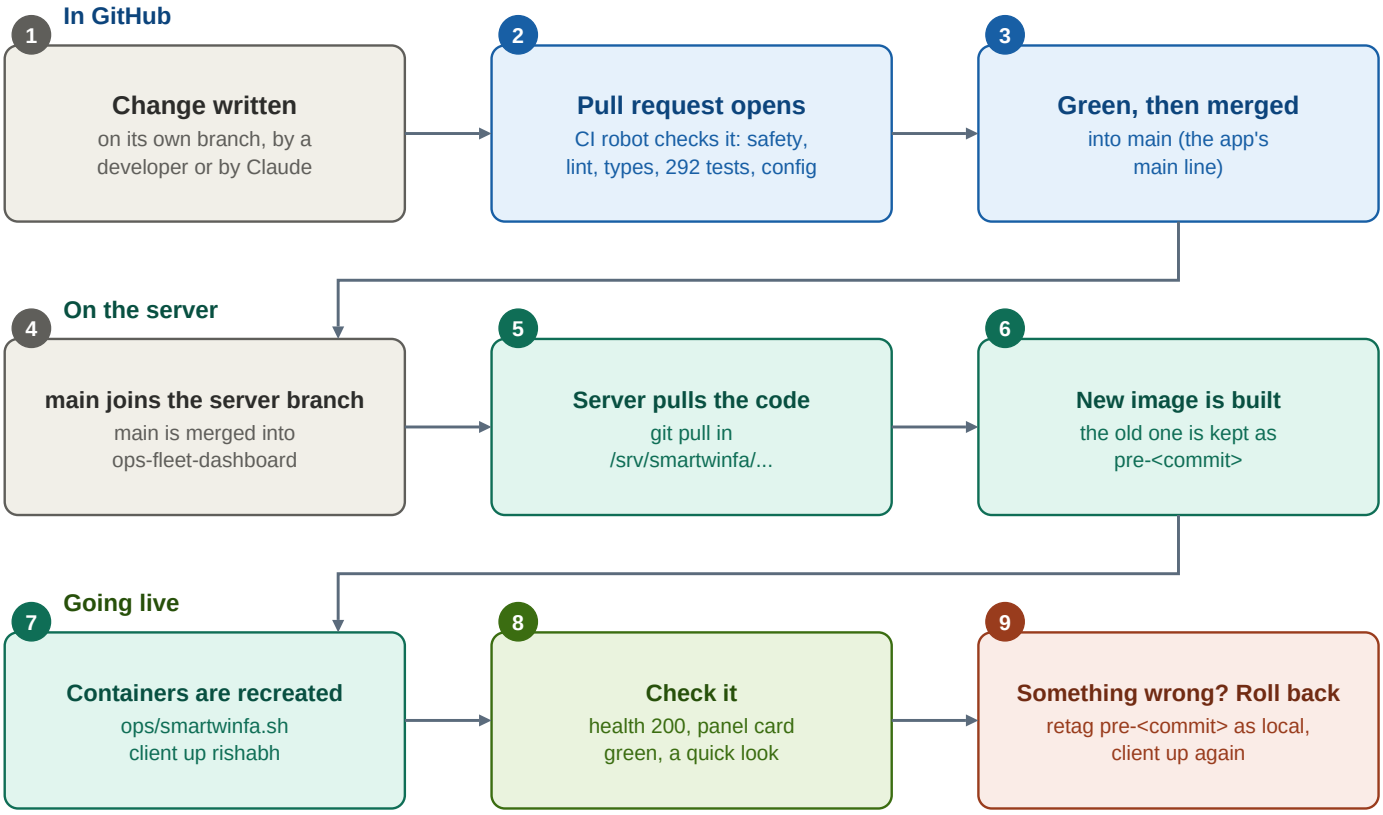
Getting a backup back

Facts

- **The first nightly backup ran on 27 October at 02:02 UTC** and was about 154 MB. The panel card turned green once it existed.
- **Time zone:** the hour is read inside the container, so it is UTC. 02:00 UTC is 07:30 in India. It can change per client with `BACKUP_HOUR` in that client's settings file; `BACKUP_KEEP_DAYS` sets how long files are kept.
- **Manual backups** are named `manual-DATE-TIME.dump` and are kept along with the nightly ones.
- **A restore replaces what is in the database.** Use it deliberately, ideally after pressing Backup now so you can go back.
- **The weak spot:** if the whole server were lost, the backups would be lost with it. Copying them to Cloudflare R2 or Backblaze B2 each hour is designed (in the site kit) and still to be switched on.

10 How a change reaches the server

Nothing is edited on the server. A change is written, checked by an automatic robot, merged, and only then pulled onto the server and rebuilt.



Why two branches

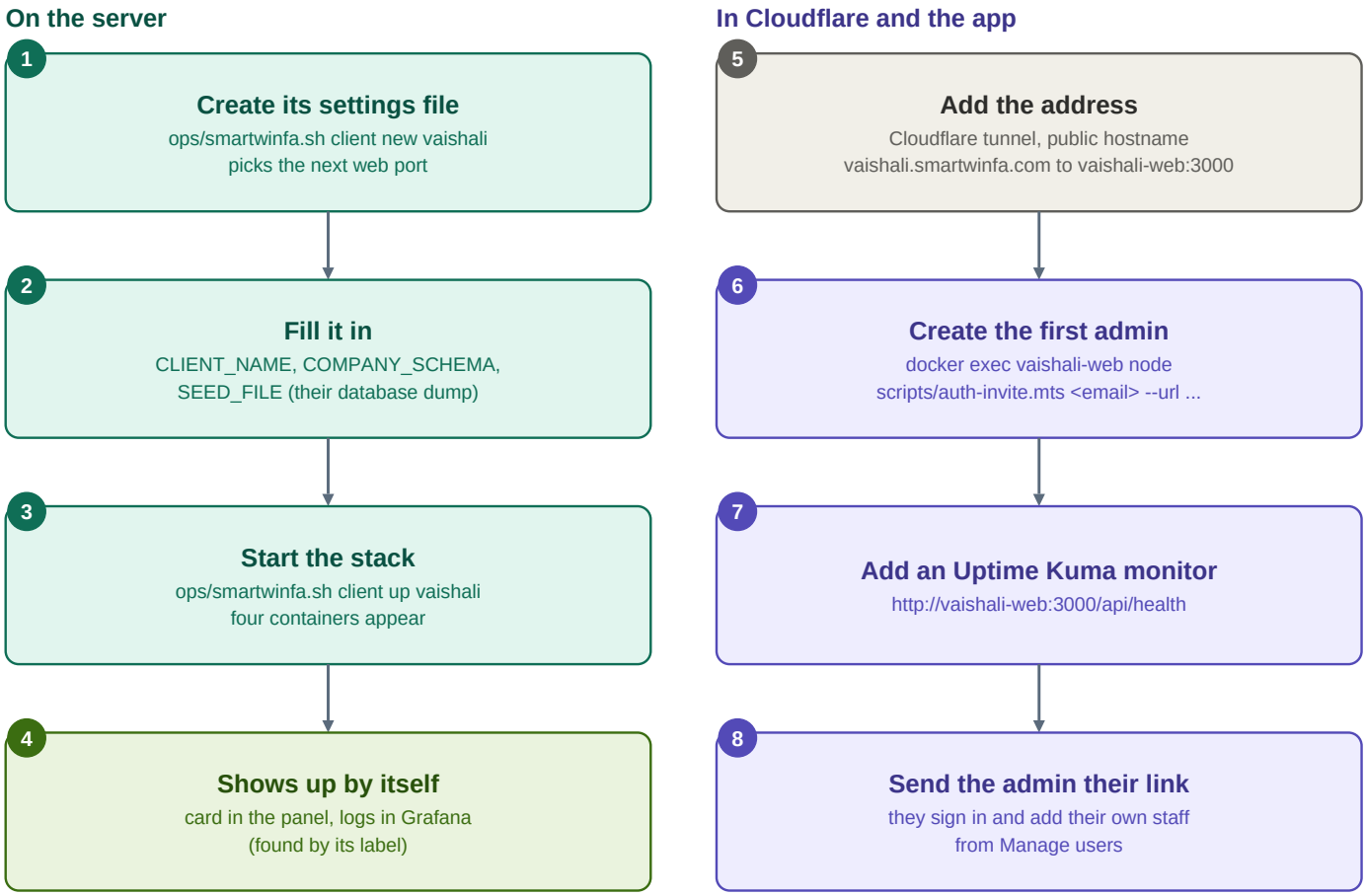
- `main` is the application: the website code and its tests. Every change to it must pass the robot.
- `ops-fleet-dashboard` is `main` plus the `ops/` folder: the scripts and settings that run the server, the logs, the panel. The server builds from this one. After each change to `main`, it is merged into this branch.

What the robot (CI) checks on every pull request

Check	Catches
Repository safety	Secrets or private files committed by mistake
Lint	Code that breaks the team's rules (unused variables, broken accessibility, risky patterns)
Type-check	Mismatches between what a function gives and what its caller expects
Build and test (292 tests)	A broken build, or behaviour that changed: sign-in, entries, reports, the panel's logic
Compose config	A mistake in the Docker setup, before it reaches the server

11 Adding another client

Eight steps, four of them on the server. After step 4 the client shows up in the panel and in Grafana without any extra work.



Capacity, in numbers

Part	Memory in use at Rishabh today
Web app	about 124 MB
API	about 89 MB
Database	about 142 MB
Backup	about 1 MB
One client	**about 360 MB**
Admin tools (panel, Grafana, Loki, Alloy, Uptime Kuma, Portainer, front door)	about 530 MB in all

This server has 3.9 GB, shared with older services, and about 650 MB was free at the last check (it also has 8 GB of swap as a safety net). A handful of clients may fit; many will not. Plan a bigger server, or a second one, before onboarding more than a few.

12 Where things live, and what to check

A cheat sheet for when you need to find something, or when something looks wrong.

What	Where
Log in to the server	`ssh -p 57289 ubuntu@5.175.139.120` (key login, from this Mac)
Code and settings	`/srv/smartwinfa/SMARTwinFA-web`, branch `ops-fleet-dashboard`
Client settings and its database password	`ops/clients/rishabh.env` (not stored in git)
Admin tool settings and passwords	`ops/fleet/.env`: `GRAFANA_ADMIN_PASSWORD`, `PANEL_PASSWORD`, `ADMIN_URL`
Portainer's password	`ops/fleet/.portainer-admin-password`
Web users of the app	In the client's own database; created from Manage users or `auth-invite`
Uptime Kuma's admin	Created by you on the first visit; kept inside its Docker volume
Backups	Docker volume `smartwinfa-rishabh_backups`
The tunnel and addresses	Cloudflare, Zero Trust, Networks, Tunnels

If something looks wrong

You see	Look here	Then
The site will not open	Panel card, then `docker ps` for smartwinfa-cloudflared	Restart app; check the tunnel in Cloudflare
Card is red	Open it: which part is down, and the error logs	Restart app; if the database is down, check disk and memory
Card is amber	Open it: errors, or no recent backup	Logs with Level = Errors; Backup now
Someone cannot sign in	Manage users: is the account disabled?	Press the link button for a fresh link
Grafana shows nothing	Widen the time range; is the client in the dropdown?	`docker logs smartwinfa-fleet-alloy-1`
The panel will not load	`docker logs smartwinfa-panel`	Sign-ins end when the panel restarts: sign in again
Server feels slow	Panel parts table: memory per part	`docker stats` on the server
Forgot a dashboard password	The files in the table above	Passwords are plain text there, readable only with sudo

13 What is done and what is open

Where things stand on 27 October 2026.

Working now

- Rishabh's app is live through the tunnel
- Web sign-in with invite links, roles and sessions
- Legacy / Modern choice remembered per browser
- Nightly backup (first one ran 27 Oct, about 154 MB)
- Logs from all four parts collected, 30 days kept
- Control panel: a card and a page per client
- Grafana, Portainer, Uptime Kuma and the panel behind one front door
- Checks on every change: lint, types, 292 tests
- Rollback images kept for each deploy

Still to do

For you

- **Add admin.smartwinfa.com** in the Cloudflare tunnel, service `http://smartwinfa-admin:80`, with a Cloudflare Access rule (staff emails and MFA).
- **Create the Uptime Kuma admin** and add the monitor for Rishabh.
- **Check last year's data:** a restore failed once during setup, so confirm the `rishabh_plastic26` tables were reloaded.
- **Choose a stronger password** for Grafana, Portainer and the panel before the address is public.

To build when you want it

- Email the set-password links from the Users screen
- Copy backups off the server (Cloudflare R2 or Backblaze B2)
- Make `/api/health` test the database, not only the web app
- A real request log from the app, so Grafana has more to show
- Images from a registry instead of building on the server
- A bigger server before onboarding many more clients

The one idea to take away

Clients are separate stacks. One tunnel brings people in. One panel, one log store and one set of backups watch them all.

When in doubt: open the panel, find the client's card, read the reason under its name, and open its page. Most questions are answered there in one click.